

基于有界测角误差与覆盖搜索的干扰源定位清除模型

摘要

针对半径1800米圆域内数量未知的无线电干扰源，建立由有界误差交会定位、覆盖搜索与光学清除组成的自动决策模型。优化目标是在完成全部清除的前提下，减少移动、频道切换、检测及光学操作的累计虚拟时间。

对于问题1，将每次示向度转化为两个线性半平面，求交得到定位区域；通过极点枚举计算直径，并区分空集、无界集及有界多边形。证明区域直径的一半不一定足以覆盖定位区域，应使用最小覆盖圆半径作为单点清除判据。对于问题2，根据接收半径下界推导由三个圆盘构成的安全候选区域。在25米网格与离散最不利精度准则下，得到沿初始示向度前进775米、横向偏移625米的对称候选方案；此数值是给定比较准则下的网格结果，不是唯一最优位置。

对于问题3，构造中心加六个环形点的七点覆盖，证明所有目标距至少一个检测点不超过900米。对于问题4，构造边长950米的三角网格，保留与目标圆域相交三角形的全部顶点，共31个检测点；利用凸组合证明任意未知定向半平面内均存在可接收检测点。局部定位采用多次测角收缩区域，在最小覆盖圆半径不超过19.99米时清除；若测角推进失败，则采用单元边长不超过27米的光学覆盖扫描，从而保留几何完整性保证。

两类问题各30次本地基准运行均清除全部目标，各累计372个，案例平均单源耗时分别为322.89秒和816.88秒。另完成压力与参数实验。对动态任务路线和部分可观测决策的有限前瞻原型开展每问200例独立配对试验，候选相对现行策略平均提速仅2.93%和0.43%，未达到预定门槛，故保留几何启发式作为最终策略。人工官方演练分别清除10/10和16/16个源；六次正式测试仍待完成，不以本地与演练结果替代正式成绩。

关键词：交会定位；有界误差；最小覆盖圆；三角网格；覆盖搜索

1 问题分析与建模目标

1.1 四个问题之间的关系

问题1解决“已经有观测，位置不确定性有多大”；问题2解决“下一次到哪里测量”；问题3和问题4进一步要求在未知频道分布和未知目标数量下，自主完成发现、定位和清除。全向场景需要空间距离覆盖，定向场景还需要方向覆盖。若先优化一条短路线，却没有证明所有目标能够被发现，就无法满足题目“确保所有干扰源被清除”的要求。

本文采用分层策略。第一层以有限检测点集提供全域发现保证；第二层利用已有示向度进行局部定位；第三层通过具有明确覆盖半径的光学操作完成清除。路线和局部选点采用启发式优化，完整性由覆盖条件与后备扫描保证。本文不声称得到全局时间最优解。

1.2 输入信息与未知量

已知参数来自赛题及附件1、附件2[1—3]。目标位置位于闭圆域内；每个源的频道固定且互异；单次示向度不提供距离、场强或发射功率。程序不能读取模拟器案例真值。无信号可能表示频道不存在，也可能表示距离或方向条件不满足。

参数	数值或规则	在模型中的作用
目标圆域半径	1800米	全局位置先验
接收半径	1000至1500米	保证发现与正观测上界
测角误差	正负1度	构造方位扇形
接口角度精度	保留两位小数	实现中额外预留0.005度
频道与源数量	20频道，10至16个源	搜索全集与停止条件
移动速度	5米每秒	距离转化为虚拟耗时
检测与切换	检测5秒，切换1秒	规划检测顺序
光学与激光	精确定位3秒，成功清除另2秒	成功或失败分别计时
光学范围	20米	覆盖圆和后备网格阈值
近场阈值	5米且位于有效角域	收到near后直接清除

1.3 目标函数与终止原则

设总移动距离为L，实际测向机频道切换次数为N_s，检测次数为N_m，光学清除尝试次数为N_c，成功清除次数为N_k，则总虚拟时间为

$$T = L / 5 + N_s + 5N_m + 3N_c + 2N_k \quad (1)$$

其中每次clear不改变测向机当前频道，失败的clear也消耗3秒。程序最终的单源平均耗时为 T/N_k ，包含发现不存在频道的检测以及终止前的完整搜索，不能只累计成功目标附近的操作。

源总数未知，清除10个不能作为终止依据。正常终止有两种证据：其一，清除16个，已达到题目给定上界；其二，完成保证覆盖点集对所有尚未清除频道的检测，并清除全部已发现目标。任何异常退出均不自动等同于完成全部清除。

2 模型假设与符号

2.1 采用的假设及其边界

严格采用题设的二维、直线移动、恒定速度、频道互不干扰和目标静止条件。不引入障碍物、移动源或信号强度观测。任一地点误差固定，重复原地检测不会产生新的独立信息，因此程序通过改变位置获取新观测。

几何保证只依赖测角误差有界，不要求其服从均匀分布。离线模拟为每个种子、位置与频道生成确定的误差，改变位置时才发生变化；它只是一种实验场景生成方式，不是对官方模拟器内部实现的推断。问题2中独立等方差小误差假设仅用于比较定位精度，不能替代几何误差界。

官方接口将示向度保留两位小数，为避免真实误差与舍入共同导致误剔除，程序采用1.005001度的保守半角。问题1理论推导仍使用题设的1度。对圆盘约束作多边形近似时，只使用外切多边形，不能用内接多边形删去可能的真实位置。

符号	含义
Ω	以原点为中心、半径1800米的目标闭圆域
g, p_i	源位置与第i个检测点
θ_i, ε	示向度与绝对角度误差上界
W_i, F	第i次方位扇形与累计可行位置区域
$D(F)$	区域内两点最大距离
c^*, r^*	最小覆盖圆圆心与半径
h, s	三角网格边长与光学方格边长
T, N_k	累计虚拟时间与成功清除总数

3 问题1 多边形直径与覆盖圆

3.1 从示向度到线性约束

记 $u(\alpha)$ 为方向角 α 的单位向量，二维叉积记为 cross 。点 x 与第 i 次观测一致，当且仅当满足

$$\begin{aligned}\text{cross}(u(\theta_i - \varepsilon), x - p_i) &\geq 0; \\ \text{cross}(u(\theta_i + \varepsilon), x - p_i) &\leq 0.\end{aligned}\quad (2)$$

两条约束的交集是朝向观测方向的闭扇形，不能改成穿过检测点的无限直线带，否则会错误保留背向源的位置。使用单位向量而不是直接比较角度区间，可自然处理0度与360度跨界。多次观测的定位区域为 F 等于所有 W_i 的交集，因而是凸集。

在问题1纯交会模型中，不能人为加一个有限方框后就宣布区域有界。算法先建立 $A \leq x \leq b$ ，对所有不平行边界线两两求交，并保留满足全部约束的交点。若无可行极点，则该组正半角方位约束不相容；若存在非零方向 d 满足 $A \cdot d \leq 0$ ，则区域沿该方向无界，直径为无穷大。本题扇形角小于180度，其交集不含直线，非空时存在极点，因此这一枚举判别适用于本模型。

非零衰退方向可从各约束边界线的两个单位切向方向中检查。计算采用容差处理平行与共点情形；算法实现给出empty、unbounded、bounded三种状态。有界时将去重后的极点按极角排序，即可得到凸多边形顶点序列。若退化为线段或点，分别返回线段长度或零。

3.2 最大距离为何只需检查顶点

任取多边形内 x 、 y ，将它们写为顶点的凸组合。由三角不等式， x 与 y 的距离不超过所有顶点对距离的最大值；而这些顶点本身属于区域。因此

$$D(F) = \max ||v_i - v_j||, \text{ 其中 } v_i、v_j \text{ 遍历所有顶点。} \quad (3)$$

设检测点数为 n 、顶点数为 m 。两两边界线相交并逐条验证的实现复杂度为 $O(n^3)$ ，顶点对直径计算为 $O(m^2)$ ，便于复核。若观测很多，可用 $O(n \log n)$ 半平面交与 $O(m)$ 旋转卡壳提升效率，但本题局部观测数量少，本文采用较简单的枚举实现。

3.3 直径为 D 的圆未必覆盖区域

答案是否定的。取边长为40米的等边三角形，区域直径为40米，而其最小覆盖圆半径为 $40/\sqrt{3}$ ，约23.094米，大于20米。因此不存在直径40米的圆覆盖这个三角形。选择最远顶点对的中点作为圆心也不能改变该反例。

实际应计算最小覆盖圆，即在全部圆心 c 中最小化 $\max_i ||v_i - c||$ 。最小覆盖圆由一个退化点、两个直径端点，或者三个非共线边界点确定。程序枚举这些候选圆心，再用全部顶点重新计算实际覆盖半径，选择最小者。三点候选通过线性方程求外接圆心；近共线三点跳过，其情形由两点候选处理。

这里 $D \leq 40$ 米只是20米单点清除的必要条件，并不充分。充分且必要的几何条件是 $r^* \leq 20$ 米。实现阈值采用19.99米，给数值舍入留出0.01米余量。如果真实可行区域含圆弧，则对它

的外包多边形计算 r^* ，所得清除条件仍然保守可靠。

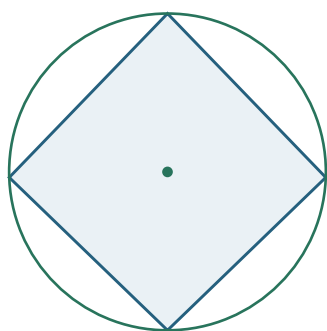
3.4 可复算数值示例

取检测点(0, 0)、(1000, 0)，示向度分别为45度、135度，误差半角为1度。得到四个顶点及结果如下，数值以geometry_summary.json为准。

顶点	x坐标 米	y坐标 米
1	482.550252	499.695414
2	500.000000	482.844387
3	517.449748	499.695414
4	500.000000	517.765157

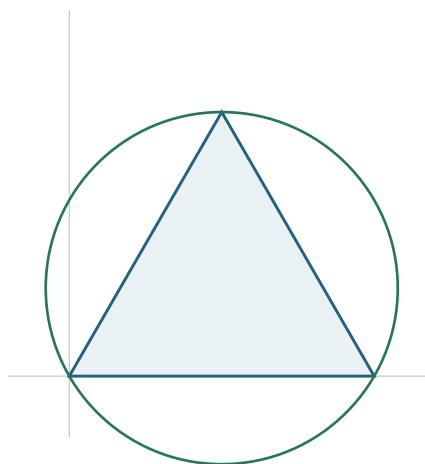
该区域直径为34.920769米，最小覆盖圆心约为(500.000000, 500.304772)，半径为17.460385米。这个具体例子恰好由一条直径确定覆盖圆，但不能推广为任意定位多边形的性质。

交会区域与最小覆盖圆



$D=34.921$ 米 $r^*=17.460$ 米

等边三角形反例



$D=40$ 米 $r^*=23.094$ 米 > 20 米

图1 交会定位区域与等边三角形反例

4 问题2 第二检测点的选择

4.1 单次方位观测的信息不足

首次示向度只给出角度扇形，不能确定目标距离，因而不存在脱离距离先验、移动成本与评价指标的唯一最佳第二点。沿原示向线前进虽能缩短距离，却可能使两条视线近乎共线，交会区域仍然很长；横向移动能改善交会角，但移动过远会失去信号。因此需要同时考虑可接收性与几何精度。

以第一次测点 p 为局部原点，首次示向度为局部正横轴。真实源可写为 $g=p+r \cdot u(\phi)$ ，其中 $|\phi - \theta| \leq \varepsilon$ ， $r \leq 1500$ 米。第一次已收到信号意味着该源的真实接收半径 R 还满足 $R \geq r$ ；题目同时给出 $R \geq 1000$ 米。因此第二点 q 满足下式，就能对所有与第一次观测相容的源保证再次接收：

$$||q - g|| \leq \max(1000, r), \text{ 对所有可行的 } r \text{ 与 } \phi \text{ 成立。} \quad (4)$$

4.2 三圆盘安全候选区

令 $B(a, R)$ 表示以 a 为圆心、半径 R 的闭圆盘。以下集合是一个可直接绘制和计算的安全候选区域：

$$K = B(p, 1000) \cap B(p+1000u(\theta - \varepsilon), 1000) \\ \cap B(p+1000u(\theta + \varepsilon), 1000)。 \quad (5)$$

证明分两段。对于 $0 \leq r \leq 1000$ ， $||q-p-r \cdot u(\phi)||^2$ 关于 r 为凸函数，只需检查 $r=0$ 和 $r=1000$ 。第一个圆盘覆盖 $r=0$ ；后两个圆盘保证角域两端的 $r=1000$ 。两端约束还保证 q 位于朝向源的前向区域， q 与 $u(\phi)$ 的点积在此小角域内的最小值发生于端点，因此内部角度也满足约束。

对于 $1000 \leq r \leq 1500$ ，由 $||q-p-1000u(\phi)|| \leq 1000$ 可得 $||q-p||^2 \leq 2000(q-p) \cdot u(\phi)$ ，且点积非负。将2000替换为更大的 $2r$ ，得到 $||q-p-r \cdot u(\phi)||^2 \leq r^2$ ，故距离不超过 $r \leq R$ 。两部分合并即证明 K 的安全性。

该候选区是保守的充分集合；还可以利用目标圆域先验、 $r > 5$ 米以及其他已知约束进一步扩充或筛选。本题允许机器狗走到目标圆域之外，因此不必把 q 强制限制在 Ω 内。 $q=p$ 虽然安全，但没有新增几何信息，应予排除。

4.3 定位精度与移动成本的权衡

用 r_1 、 r_2 表示两个检测点到真实源的距离， γ 表示两条视线交角。对小角度误差线性化，在两次误差独立、等方差 σ^2 这一额外比较假设下，二维位置协方差的迹近似为

$$\text{tr}(C) = \sigma^2 (r_1^2 + r_2^2) / \sin^2 \gamma。 \quad (6)$$

推导方式为将每次角度扰动转化为与视线垂直的横向扰动 $r_i \delta \theta_i$ ，再求解两条线性横向约束。交角接近0度或180度时分母接近零，位置误差放大；距离和交角必须共同衡量，不能简单断言所有情况下90度最优。若没有统计独立性，式(6)只能作为选点启发式，不能作为置信区间。

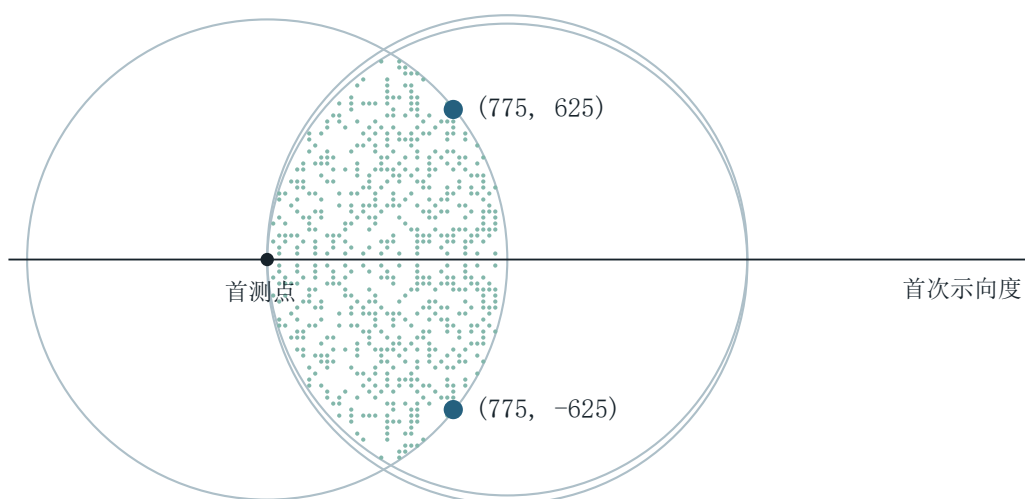
本文在 K 内以25米为步长生成候选点，剔除横向偏移为零的退化点；以151个径向位置和9个角度位置离散初始扇形，对 $\sqrt{r_1^2 + r_2^2} / |\sin \gamma|$ 取最大值，选择离散最不利得分较小的点。上半平面共944个候选点，下半平面为镜像。在这些离散条件下，最优网格点为局部坐标(775, 625)米或(775, -625)米，得分约2729.782米每弧度。

这个候选点到第一次测点的距离约995.62米，移动耗时约199.12秒；原频道不变时第二次检测另需5秒。若额外取均匀正负1度误差， $\sigma = \pi / (180 \sqrt{3})$ ，离散最不利线性化均方根指标约27.51米。该数字不是硬误差界，也不说明两次观测一定已达到20米清除条件，仍须计算实际

交会区域。

若强调总任务时间，可在候选点得分中加入移动成本，或对移动距离设置预算。在问题3、4的程序中，我们采用较短的局部收缩步，而不是每个目标固定移动约996米。问题2的安全区域和比较算法提供第二点的可解释设计依据，局部动态策略则根据累计观测自适应调整。

三个圆盘交集内的离散安全候选



单位 米 绿色点为候选区 深蓝点为所用准则的网格解

图2 第二检测点安全区域与离散候选

5 通用局部定位与清除算法

5.1 正观测累积与区域收缩

对每个频道分别保存位置与示向度序列。新的正观测加入后，将原区域与新的方位扇形、以当前检测点为圆心的1500米圆盘相交，同时保留全局目标圆域约束。程序将目标圆域近似为32边外切多边形，将接收圆盘近似为16边外切多边形；所有支持直线均朝外，故不会排除符合物理条件的真实源。

每轮计算外包区域的最小覆盖圆。如果 $r^* \leq 19.99$ 米，在以 c^* 为中心、半径 $19.99 - r^*$ 的安全圆盘内选择距当前位置最近的点 x 执行clear。对任意可行源 g ， $\|x - g\| \leq \|x - c^*\| + r^* \leq 19.99$ 米，故该位置仍有清除保证。收到near时直接在当前位置执行clear，因为5米小于20米。如果一个被证明安全的clear却失败，或正观测约束交集为空，程序停止并报告模型或协议不一致，不把矛盾解释为正常无信号，也不扩大阈值伪装成功。

5.2 自适应观测步

问题3同时考虑当前位置与最新成功测点 p_0 ，生成沿区域中心方向前进0.5、0.8、1.0比例及横移的候选，保留0.65比例的原候选。用有限区域样本估计测角后的区域半径、接收可能性和后续移动，选择经验代价较小的候选。样本仅用于排序，实际几何区域仍使用全部角度约束。

设 $d=c*-p_0$ ， v 为 d 的垂直单位向量。问题4沿用经过演练的候选构造：

$$q = p_0 + 0.8d \pm \min(100, 0.35||d||)v。 \quad (7)$$

问题4优先选择距当前位置较近的一侧；必要时先在已移动到的位置补测。若一侧无信号则尝试另一侧，两侧均无信号或达到8轮上限时进入光学后备。问题3遇到连续两次无信号或达到8轮上限也进入后备。无信号不会从严格几何集合中删除源位置。

这些经验候选不保证时间最优，也不承诺每一步必有信号。第8.7节进一步比较信念前瞻原型并如实报告未采用的结果。重复原地检测不作为独立误差平均使用；有限迭代和完整光学后备共同避免无限测量。

5.3 光学后备扫描及有限性证明

将当前外包多边形转到首次示向度坐标系，取轴平行外接矩形。沿两个方向均匀分格，使每个单元的两条边不超过27米。在每个单元中心执行clear；从完整单元集合构造最近邻开放路线，再作至多三轮2-opt边交换。所有单元中心均保留，优化只改变访问顺序。

任一点到所在单元中心的距离不超过 $27/\sqrt{2}$ ，约19.092米，小于20米。真实源属于外包多边形，必属于某个单元，所以在扫描完全部单元之前一定会被清除。这个论证不依赖发射朝向，正好补足定向信号丢失时测角无法推进的问题。

首次正观测还给出最大接收距离上界。16边外切圆盘的外接半径不超过 $1500/\cos(\pi/16)$ ，约1529.39米；小角度方位扇形对应的横向总宽度小于54米。因此仅有首次观测时，外接矩形最多约57列、2行，共114个光学单元，随后观测只会缩小矩形。后备阶段是有限的，远小于盲扫整个目标圆域的光学操作量。

6 问题3 全向场景的完整搜索

6.1 七点覆盖构造

选中心点 $p_0=(0,0)$ ，以及半径 $\rho=1800\cos 30^\circ$ 、方位角依次为0、60、120、180、240、300度的六个环形点。

检测点	x坐标 米	y坐标 米
0	0.000	0.000
1	1558.846	0.000

检测点	x坐标 米	y坐标 米
2	779.423	1350.000
3	-779.423	1350.000
4	-1558.846	0.000
5	-779.423	-1350.000
6	779.423	-1350.000

证明如下。半径 $r \leq 900$ 米的点距中心不超过900米。对于 $900 \leq r \leq 1800$ 米的点，取角度最近的环形检测点，夹角 $\delta \leq 30$ 度，两点距离平方不超过 $r^2 + \rho^2 - 2r\rho \cos 30$ 度。该式关于 r 为凸函数，在区间两个端点均等于 900^2 ，因此整个区间不超过 900^2 。七个半径900米圆盘覆盖 Ω ，而真实接收半径至少1000米，存在100米覆盖余量。

6.2 搜索顺序与频道调度

机器狗从原点出发，在覆盖站点连续扫描所有未清除频道，优先扫描当前频道。已发现源进入待处理队列；其区域中心与尚未完成的覆盖站点一起构成临时任务点集，用最近邻和至多三轮2-opt规划开放路线，每次只执行第一个任务。

若选择覆盖站点，则在那里扫描并更新所有已知频道的区域；若选择目标，则定位并清除。每个目标最多延后三个覆盖站点，达到上限后强制处理；覆盖完成后清空待处理队列。一个源被清除后永久移出扫描集合。目标暂时无信号不会使其从队列丢失，程序也不要求返回原点。

6.3 完整性与时间含义

考虑任意未清除全向源。七点覆盖保证至少存在一个检测点能接收它。如果处理到该点之前它还没被清除，该频道就仍在扫描集合中，因此能够被发现；被发现后保留在有限延期队列中，局部程序具有有限清除保证。对全部源应用上述论证，得到全域完整性。

该证明与访问顺序无关。当前位置随定位过程改变，单独求静态七点旅行商路线并不等价于最小化总任务时间；本文采用动态任务排序，并另用同场景对照评估更复杂的服务成本和信念前瞻改动。

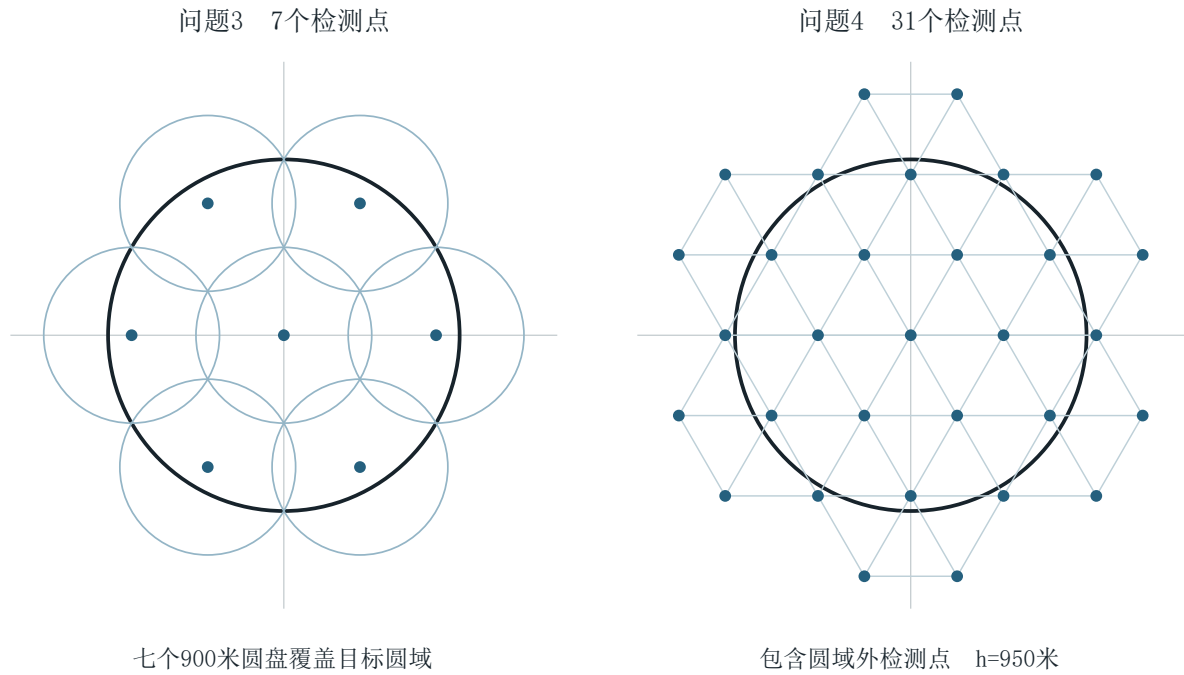


图3 全向七点覆盖与混合场景三角网格

7 问题4 含定向源的完整搜索

7.1 为什么全向覆盖不足

即使某点距离源很近，也可能位于定向源的背向半平面。尤其当源在目标圆域边界向外辐射时，仅在圆域内部布点无法保证检测到它。附件允许机器狗提交圆域外坐标，因此必须使用这一条件构造方向覆盖，不能把“无信号”简单视为空间空白。

7.2 三角网格与角域覆盖证明

令 $a=(h, 0)$ 、 $b=(h/2, \sqrt{3}h/2)$ ，取整系数格点 $i a + j b$ ，将每个平行四边形拆成两个等边三角形。保留所有与 Ω 相交的闭三角形，并收集其全部顶点。取 $h=950$ 米时得到 31 个检测点；圆域外的点是该保证构造的一部分。

任取源位置 g ，它属于至少一个被保留的三角形，可表示为三个顶点 p_i 的凸组合，即 $g = \sum \lambda_i p_i$ ， $\lambda_i \geq 0$ 且 $\sum \lambda_i = 1$ 。设定向半平面的单位朝向为 n 。如果所有顶点都严格位于背向半平面，则 $n \cdot (p_i - g) < 0$ ，将其按 λ_i 加权求和得到 $0 < 0$ ，矛盾。因此至少有一个顶点位于有效的闭半平面。

另一方面，三角形内部点到任一顶点的距离不超过边长 $h \leq 1000$ 米。故上述顶点同时满足距离和方向条件，一定可接收信号。若源恰在格点上，为避免零距离方位的实现歧义，可考虑围绕该格点的六个邻点；这些点也被闭相交规则纳入，其中总有非零向量位于任何给定半平面内，距离为 h 。格边上的情况同理由邻接三角形覆盖。

7.3 调度与定位

问题4优先访问圆域内尚未完成的覆盖站点，再访问外围站点；各层内部采用最近邻与2-opt开放路线。每个站点扫描后及时处理该批目标，批次顺序考虑后续覆盖站点作为路线衔接点。全体31个覆盖站点仍被保留；清除16个源时可依据数量上界提前结束。局部无信号既不删除位置也不删除频道，而是尝试另一侧测点或进入光学扫描。

有限检测点对每个源保证至少一次发现；后备光学覆盖保证每个已发现源最终清除；因此在题设物理规则和正常执行条件下，两层保证组合为混合场景完整性。真实联网故障、程序现实时间截止和模拟器异常属于执行约束，不能由几何证明自动消除，需通过官方演练确认通信和实际运行时限。

8 数值实验与结果

8.1 实验来源与复现设置

本节全部数值来自独立编写的LocalBackend，不是题目官方模拟器的演练或正式测试。控制器只调用enter、measure、clear、exit，不访问源坐标、半径、朝向和总数；测试框架仅在运行结束后读取本地真值核对清除比例。

基准实验分别为问题3、4生成30个场景，种子为整数20260910至20260939。数量在10至16之间离散均匀抽取，频道无放回抽取，位置在圆域按面积均匀抽取，接收半径在1000至1500米均匀抽取。问题4的每个源以一半概率设为定向，朝向在全圆均匀抽取。误差由种子、频道和位置散列生成，原地重复时保持不变。

以上是为实验明确声明的生成规则，不能据此推断官方案例分布。相同种子的参数实验复用场景以便比较，不是新的独立样本。运行环境为Python 3.12.14、NumPy 2.3.5；本地程序时间不包含网络往返和官方界面处理开销。

8.2 基准统计

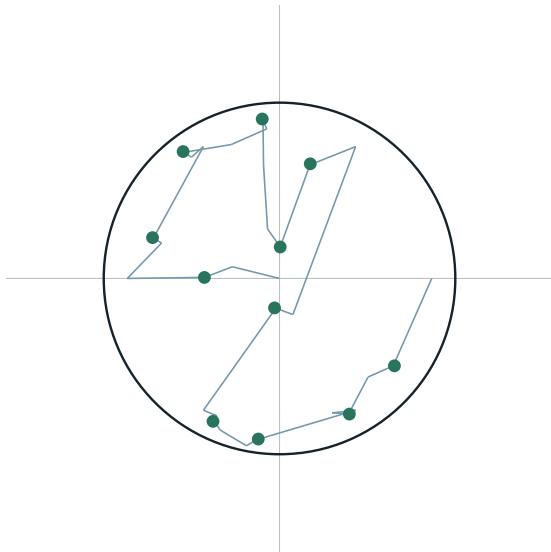
每个案例首先计算 T_j/N_j ，再对30个案例作算术平均。另报告 $\Sigma T_j/\Sigma N_j$ 作为源数量加权平均，避免混淆两种统计口径。

本地基准指标	问题3	问题4
运行次数	30	30
累计真实源数	372	372
累计清除源数	372	372
每次清除比例	100%	100%
案例平均单源时间 秒	322.89	816.88
案例单源时间样本标准差 秒	34.33	125.42

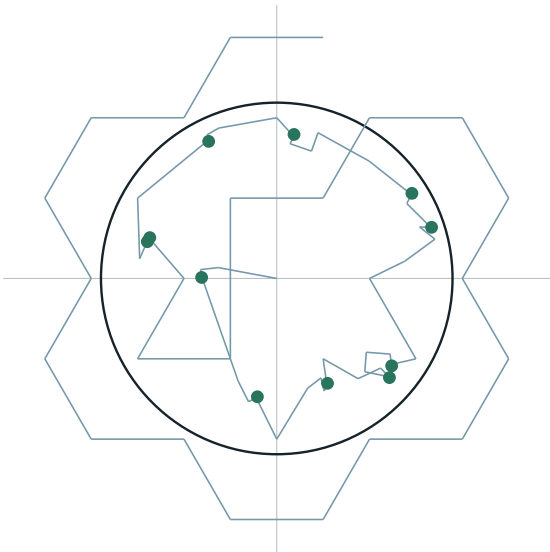
本地基准指标	问题3	问题4
源数量加权平均时间 秒	318.94	798.89
最大本地程序运行时间 秒	0.271	0.637

混合场景耗时较大，一方面因为方向覆盖需要更多检测点，另一方面因为部分局部测点落在背向区域，触发额外检测或光学扫描。问题3基准没有触发后备扫描；这不能证明实际全向案例永远不需要它，因此代码仍然保留完整后备流程。

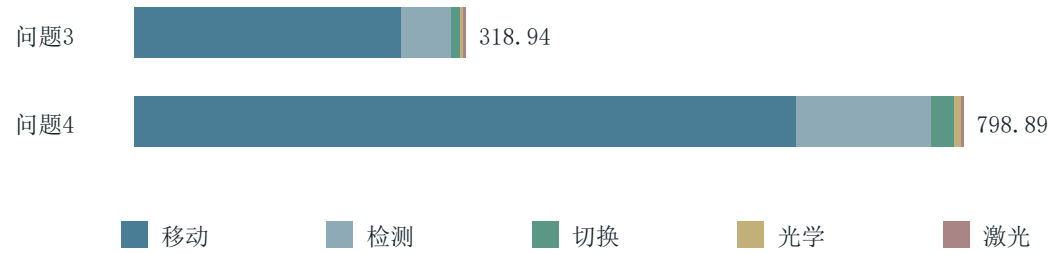
本地问题3示例轨迹



本地问题4示例轨迹



按清除总数加权的耗时组成 秒每个源



全部为本地生成场景结果 非官方测试

图4 本地示例轨迹与耗时组成

8.3 极端误差与边界压力测试

将所有接收半径固定为最小值1000米，分别设置各地点误差恒为正1度或负1度；混合场景进一步把所有源都设成定向。另构造20个边界场景，全部源位于半径1800米圆周，朝向在向外和切向两类设置间切换。结果如下。

本地压力设置	次数	清除数与总数	案例平均单源秒
问题3 正1度 最小半径	20	244 / 244	324.23
问题3 负1度 最小半径	20	244 / 244	325.57
问题4 全定向 正1度 最小半径	20	244 / 244	845.68
问题4 全定向 负1度 最小半径	20	244 / 244	836.78
问题4 圆周向外或切向	20	259 / 259	860.69

这些结果支持后备机制在所测困难条件下可用。完整性的主要依据仍然是第6、7节的覆盖证明，而不是有限次数实验中的100%。

8.4 网格间距敏感性

在同一组20个问题4场景上改变h，保持其他策略一致。950米网格在这一组对照中较好，因此保留为默认值；并未据此声称它在所有场景或所有可能网格中最优。

h 米	覆盖点数	清除数与总数	案例平均单源秒
800	37	251 / 251	815.51
900	37	251 / 251	886.02
950	31	251 / 251	781.15
999	31	251 / 251	786.44

间距并非越大越快。点数变化是离散的，最近邻路线和已发现源的空间顺序也会变化；较远检测还可能使局部误差区域增大。因此总时间应通过同场景对照计算，而不是只看覆盖点数。

8.5 局部测角的消融对照

将局部测角迭代上限设为0，问题3直接进入几何清除或光学覆盖；问题4保留到位后的一次补测，但不再执行迭代测角。该消融不是绝对零测量策略。在每类问题前10个基准场景上，两类各127个目标均成功清除。完整策略与消融策略使用相同案例比较，结果如下，不能以30例均值替代配对均值。

同场景对照	完整策略 秒每源	禁用迭代测角 秒每源	均值降低比例
问题3 前10例	326.77	562.13	41.87%
问题4 前10例	793.25	968.79	18.12%

该对照说明光学覆盖可以独立承担清除保证，但测角收缩通常能减少需要访问的光学单元。后备策略的价值在于应对局部观测失败，而不宜把其高覆盖成本作为每个目标的默认流程。

8.6 程序验证与误差防护

16项自动检查均通过，覆盖附件199秒计时示例、clear不改变频道、固定地点误差不变、0度跨界和舍入误差、空/无界/有界区域识别、等边三角形覆盖反例、七点距离覆盖、三角网格半平面覆盖、强制光学后备、背向光学清除及请求丢失后的幂等重试。

其中网格采样检查不能取代连续覆盖证明。HTTP单元检查验证请求体与request_id完全复用；另有人工官方演练的逐步记录用于核对真实协议和计时。源码统一前后在200个普通、边界、切向、聚集和相关误差场景中逐条动作一致，全部清除，共400次运行。

8.7 动态任务路线与信念前瞻的对照

将搜索、定位和清除表示为动态任务，研究将未来频道检测成本纳入开放路线；另采用对位置、半径和方向的假设样本，按观测更新并进行两步局部模拟，借鉴部分可观测决策与rollout方法[5-7]。原型不是完整POMDP求解，未知源数量估价使用期望13的简化，先验不代表官方真实分布。几何完整性不依赖样本权重。

开发阶段每问60个场景，种子720000至720059，分别比较路线、信息决策及联合原型。源码和门槛在独立留出前冻结，每问200个新场景，种子820000至820199；问题3选择联合候选，问题4选择开发结果较好的路线候选。每个场景由初始基线、现行策略和候选分别运行，共1200次运行，全部清除。

留出对照 秒/源	问题3	问题4
现行策略均值	320.360	772.857
候选均值	310.987	769.548
现行策略P95	393.002	990.314
候选P95	385.158	994.556
候选平均提速	2.93%	0.43%
最差单例耗时比	1.309	1.578
是否采用新增候选	否	否

采用门槛为平均至少提速5%、P95不增加、慢超过10%的场景不超过5%且最差耗时比不超过1.20。候选未达门槛，因此论文最终算法及运行入口保留现行策略，不以更复杂模型的名称作为效率证据。所有退化案例与源码测试夹具保存在支撑材料中。

9 官方测试记录与接口执行

9.1 测试状态

现行策略已由人工完成官方演练：问题3案例M2UK-D2TX-MRQJ-7P76清除10/10个源，平均406.723秒/源；问题4案例4T23-B68S-PBKS-AB2C清除16/16个源，平均360.112秒/源，两次均正常退出。已核对平台实际源数、客户端响应和导出日志公开头。这些是演练记录，未填入正式成绩表。

本轮源码统一后未新增官方测试，采用离线动作一致性验证整合。六次正式测试尚无完成记录，以下保留待测试字段，不填入推测成绩。

表1 问题3正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
第1次 待测试	待测试	待测试	待测试
第2次 待测试	待测试	待测试	待测试
第3次 待测试	待测试	待测试	待测试

表2 问题4正式测试结果

测试案例编码	清除干扰源个数	平均定位清除时间	程序运行时间
第1次 待测试	待测试	待测试	待测试
第2次 待测试	待测试	待测试	待测试
第3次 待测试	待测试	待测试	待测试

9.2 串行接口与时间控制

程序使用标准库HTTP+JSON客户端，arena_id固定为default，robot_id由运行参数提供且必须与登录参赛队号相同。每个新动作生成唯一request_id，通信中断重试保持路径、请求体和request_id不变。只在accepted=true时更新位置与虚拟时钟，不把拒绝响应中的virtual_time_s=0当作当前时间。

enter成功后读取remaining_real_duration_s设置现实截止，不能固定假定每局有1200秒。检测5秒是虚拟耗时，现实中不需要额外睡眠5秒。各动作依次等待响应，不并发发送不同动作。未确认通信故障不会触发新的不同动作，正常全部完成才调用exit。

9.3 补齐正式结果的步骤

在模拟器内登录后，先使用对应问题的演练模块执行run.py的official模式；这个模式表示连接官方HTTP端口，不会自动选择或启动正式场次。演练结束读取实际总数，核对清除比例、时间和日志大小。经过充分官方演练后，再分别在两个正式模块完成三次测试。

每次记录界面提供的案例编码及最终结果，将模拟器导出的加密日志以原文件名放入 official_logs。客户端JSONL是辅助记录，不能替代正式加密日志。最后以实际记录更新本节、摘要与相关结论，并重新核对上传与支撑材料。题面规定的测试启动截止为北京时间2026年9月13日17时30分，建议15时30分前完成正式测试。

10 模型评价与结论

本模型的主要优点是区分了“误差区域小”和“一个20米圆能够覆盖”两种不同条件；为第二检测点提供有证明的接收安全区域；将全向距离覆盖扩展为定向半平面覆盖；并通过有限光学扫描处理局部定位失败。模型不依赖隐藏距离或场强信息，程序与题目四个接口直接对应。

主要限制是时间优化采用启发式策略，尚未求出最优值或最优差距；问题2的数值候选是离散比较结果；三角网格的完整性以允许走出目标圆域为前提；本地生成误差和场景无法复现未知的官方分布。演练已验证过正常联网执行，但目前论文仍缺少题目要求的六次正式成绩及对应正式加密日志。

动态任务路线和信念前瞻原型已进行对照，其有限收益和退化说明概率模型与末端成本估计仍需改进。未通过门槛的原型不作为默认策略，继续研究需使用新的独立验收场景。当前方案已经给出四问一致的数学框架、可复现程序和本地验证证据，正式提交前仍需完成第9节的真实测试。

参考文献

- [1] 2026年高教社杯全国大学生数学建模竞赛 B题 无线电干扰源的快速自动定位与清除，用户提供的B题.pdf，2026。
- [2] B题附件1 模拟器使用说明，用户提供的附件1.docx，2026。
- [3] B题附件2 模拟器通信接口说明及编程指南，用户提供的附件2.docx，2026。
- [4] 全国大学生数学建模竞赛组委会 全国大学生数学建模竞赛论文格式规范 2026年修订稿，2026年3月3日，https://www.mcm.edu.cn/html_cn/node/4cd596519c9eb9fbd866398f6df0caa3.html，访问日期2026年9月10日。
- [5] Kaelbling L P, Littman M L, Cassandra A R. Planning and acting in partially observable stochastic domains. Artificial Intelligence, 1998, 101: 99-134.
- [6] Silver D, Veness J. Monte-Carlo Planning in Large POMDPs. Advances in Neural Information Processing Systems, 2010, 23.
- [7] Bertsekas D P, Tsitsiklis J N, Wu C. Rollout Algorithms for Combinatorial Optimization. Journal of Heuristics, 1997, 3: 245-262.

附录A 支撑材料与复现说明

最终日常运行入口统一为support/run.py，支撑材料含几何、求解器、后端、计时核查、问题2计算、实验与测试程序，以及results中的实际结果。试验原型只作为JSON实验源码夹具保存，不作为可选择的日常程序。官方原始记录另存演练记录目录，保留原文件名；含身份字段的客户端原始记录不直接打入匿名支撑材料。

在support目录执行python -m unittest -v test_model test_controller运行16项检查；执行python question2.py重算问题1示例和问题2候选；执行python run.py --problem 3 --runs 30与对应问题4命令复算基准；执行python experiments.py重算压力、参数和消融。verify_experiments.py可在临时目录从测试夹具复现留出对照。日常程序没有版本选择参数。

AI工具参与了题目提取、模型推导、程序编写、测试设计及论文排版。文中理论和数值需由使用者理解、复核，并按适用要求如实说明工具使用。没有人工签署承诺书，也没有代填参赛身份信息。

附录B 完整源程序

geometry.py

```
"""Conservative bearing geometry. Coordinates metres, angles degrees."""
import math
import itertools
import numpy as np
EPS_DEG = 1.005001 # 1 degree physical error plus 0.005 degree rounding
def unit(deg):
    a = math.radians(deg)
    return np.array([math.cos(a), math.sin(a)])
def clip(poly, normal, bound):
    """Clip convex polygon by normal @ x <= bound, retaining boundary."""
    if len(poly) == 0:
        return np.empty((0, 2))
    out = []
    prev = poly[-1]
    fp = float(np.dot(normal, prev) - bound)
    for cur in poly:
        fc = float(np.dot(normal, cur) - bound)
        if (fp <= 1e-8) != (fc <= 1e-8):
            out.append(prev + fp / (fp - fc) * (cur - prev))
        if fc <= 1e-8:
            out.append(cur)
        prev, fp = cur, fc
    if not out:
        return np.empty((0, 2))
    clean = [out[0]]
    for p in out[1:]:
        if np.linalg.norm(p-clean[-1]) > 1e-7:
            clean.append(p)
    if len(clean)>1 and np.linalg.norm(clean[0]-clean[-1])<1e-7:
        clean.pop()
    return np.asarray(clean)
def bearing_clip(poly, position, angle, error=EPS_DEG):
    p = np.asarray(position, dtype=float)
    lo, hi = unit(angle-error), unit(angle+error)
    # cross(lo, x-p)>=0 and cross(hi,x-p)<=0
    for n in (np.array([lo[1], -lo[0]]), np.array([-hi[1], hi[0]])):
        poly = clip(poly, n, float(n @ p))
    return poly
def feasible(observations, error=EPS_DEG):
    """Outer polygon, never an inscribed approximation of a physical disk."""
```

```

poly = np.array([[ -1800., -1800.], [1800., -1800.], [1800., 1800.], [ -1800., 1800.]])
for deg in np.arange(0., 360., 11.25):
    poly = clip(poly, unit(deg), 1800.)
for p, angle in observations:
    poly = bearing_clip(poly, p, angle, error)
    for deg in np.arange(0., 360., 22.5):
        n = unit(deg)
        poly = clip(poly, n, float(n@p)+1500.)
return poly
def diameter(poly):
    if len(poly)==0:
        raise ValueError('Empty region has no localization diameter')
    delta = poly[:, None, :] - poly[None, :, :]
    d2 = (delta*delta).sum(axis=2)
    i, j = np.unravel_index(np.argmax(d2), d2.shape)
    return math.sqrt(float(d2[i, j])), (poly[i], poly[j])
def region_from_bearings(observations, error=1.):
    """Q1 only: exact wedge intersection, with empty/unbounded classification.
    No artificial bounding square or reception disk is added in this function.
    Pair enumeration  $O(n^3)$ ; diameter over finite vertices  $O(v^2)$ .
    """
    if not observations:
        return {'status': 'unbounded', 'diameter': float('inf'), 'vertices': np.empty((0, 2))}
    normals = []; bounds = []
    for p, angle in observations:
        p = np.asarray(p)
        lo, hi = unit(angle - error), unit(angle + error)
        for n in (np.array([lo[1], -lo[0]]), np.array([-hi[1], hi[0]])):
            normals.append(n); bounds.append(float(n@p))
    A = np.array(normals); b = np.array(bounds)
    vertices = []
    for i, j in itertools.combinations(range(len(A)), 2):
        mat = A[[i, j]]
        if abs(np.linalg.det(mat)) < 1e-12:
            continue
        p = np.linalg.solve(mat, b[[i, j]])
        if np.all(A@p <= b+1e-7) and all(np.linalg.norm(p-q) > 1e-6 for q in vertices):
            vertices.append(p)
    if not vertices:
        return {'status': 'empty', 'diameter': None, 'vertices': np.empty((0, 2))}
    for n in A:
        for d in (np.array([-n[1], n[0]]), np.array([n[1], -n[0]])):
            if np.all(A@d <= 1e-10):
                return {'status': 'unbounded', 'diameter': float('inf'), 'vertices': np.array(vertices)}
    vertices = np.array(vertices)
    center = vertices.mean(axis=0)
    order = np.argsort(np.arctan2(vertices[:, 1]-center[1], vertices[:, 0]-center[0]))
    vertices = vertices[order]
    return {'status': 'bounded', 'diameter': diameter(vertices)[0], 'vertices': vertices}
def enclosing_circle(poly):
    """Exact candidate enumeration; final radius recomputed over every vertex."""
    if not len(poly):
        raise ValueError('Inconsistent or empty bearing intersection')
    best_c = np.mean(poly, axis=0)
    best_r = float(np.max(np.linalg.norm(poly-best_c, axis=1)))
    def consider(c):
        nonlocal best_c, best_r
        r = float(np.max(np.linalg.norm(poly-c, axis=1)))
        if r < best_r:
            best_c, best_r = c, r
    for p in poly:
        consider(p)
    for a, b in itertools.combinations(poly, 2):
        consider((a+b)/2)
    for a, b, c in itertools.combinations(poly, 3):
        mat = 2*np.array([b-a, c-a])
        if abs(np.linalg.det(mat)) < 1e-9:
            continue
        center = a + np.linalg.solve(mat, np.array([np.dot(b-a, b-a), np.dot(c-a, c-a)]))
        consider(center)
    return best_c, best_r

```

```

def optical_cover(poly, angle):
    """Cover enclosing rotated rectangle by cells of side <=27, radius <20."""
    u=unit(angle); v=np.array([-u[1],u[0]])
    axes=np.array([u,v])
    q=poly@axes.T
    lo=q.min(axis=0); hi=q.max(axis=0)
    counts=np.maximum(1,np.ceil((hi-lo)/27.).astype(int))
    step=(hi-lo)/counts
    out=[]
    # snake along long dimension; only <=2-3 rows for a single bearing cone
    for j in range(counts[1]):
        ix=range(counts[0]) if j%2==0 else range(counts[0]-1,-1,-1)
        for i in ix:
            out.append((lo+step*np.array([i+.5,j+.5]))@axes)
    return np.asarray(out)

def distance_origin_triangle(t):
    crosses=[]
    for i in range(3):
        a=t[(i+1)%3]-t[i]; b=-t[i]
        crosses.append(float(a[0]*b[1]-a[1]*b[0]))
    if min(crosses)>=-1e-9 or max(crosses)<=1e-9:
        return 0.
    ds=[]
    for a,b in zip(t,np.roll(t,-1,axis=0)):
        d=b-a
        s=np.clip(-float(a@d)/float(d@d),0,1)
        ds.append(np.linalg.norm(a+s*d))
    return float(min(ds))

def search_stations(problem, spacing=950.):
    if problem==3:
        radius=1800*math.cos(math.pi/6)
        return np.array([[0.,0.]]+[list(radius*unit(60*k)) for k in range(6)])
    if not 0<spacing<=1000:
        raise ValueError('Directional coverage requires 0 < spacing <= 1000')
    a=np.array([spacing,0.]); b=np.array([spacing/2,spacing*math.sqrt(3)/2])
    k=math.ceil(2*1800/spacing)+3
    vertices=set()
    for i in range(-k,k+1):
        for j in range(-k,k+1):
            base=i*a+j*b
            for offsets in ([np.zeros(2),a,b],[a,b,a+b]):
                t=np.array([base+z for z in offsets])
                if distance_origin_triangle(t)<=1800+1e-8:
                    vertices.update(tuple(np.round(p,9)) for p in t)
    return np.array(sorted(vertices))

```

solver.py

```

"""Single observation-only controller for both problems.
Coverage and geometric certificates provide completeness; routing is heuristic.
"""
import time, math
import numpy as np
from geometry import feasible,enclosing_circle,optical_cover,search_stations,bearing_clip,EPS_DEG
def safe_clear_point(current,center,radius):
    """Projection onto a sufficient safe disk; radius error margin retained."""
    if radius>19.99:
        raise ValueError('Region cannot be certified by a 19.99m disk')
    d=np.asarray(current)-center
    distance=np.linalg.norm(d)
    slack=max(0.,19.99-radius)
    return np.asarray(current).copy() if distance<=slack else center+d*(slack/max(distance,1e-15))
def open_route(points,start,end=None):
    """Nearest-neighbor seed and bounded 2-opt; optimize estimated open path."""
    if not len(points):return []
    points=np.asarray(points)
    pending=list(range(len(points)));order=[];p=np.asarray(start)
    while pending:
        j=min(pending,key=lambda i:np.linalg.norm(points[i]-p))

```

```

        order.append(j);pending.remove(j);p=points[j]
def edge(a,b):return float(np.linalg.norm(a-b))
# At most three improving passes, independent of the simulator speed.
for _ in range(3):
    improved=False
    for i in range(len(order)-1):
        before=start if i==0 else points[order[i-1]]
        for j in range(i+1,len(order)):
            after=points[order[j+1]] if j+1<len(order) else end
            old=edge(before,points[order[i]])
            new=edge(before,points[order[j]])
            if after is not None:
                old+=edge(points[order[j]],after)
                new+=edge(points[order[i]],after)
            if new+1e-7<old:
                order[i:j+1]=reversed(order[i:j+1]);improved=True
    if not improved:break
return order
class Solver:
    def __init__(self,backend,problem,spacing=950.,max_refine=8):
        self.api=backend;self.problem=problem;self.max_refine=max_refine
        self.stations=search_stations(problem,spacing)
        self.pos=np.zeros(2);self.channel=1;self.cleared=set()
        self.obs={c:[] for c in range(1,21)}
        self.counts={'measure':0,'clear':0,'failed_clear':0,'fallback':0}
        self.visited=[];self.deadline=float('inf')
        self.cache={};self.measured={c:[] for c in range(1,21)}
        self.pending_stations=[];self.deferred={};self.max_defer_stations=3
        self.parts={'move_s':0.,'switch_s':0.,'measure_s':0.,'optical_s':0.,'laser_s':0.}
        self.diagnostics={'candidate_evaluations':0,'deferred_targets':0,'forced_targets':0,
            'opportunistic_measures':0,'safe_clear_points':0}
        self.certificates=[]
    def action(self,path,p,c):
        if time.monotonic(>)self.deadline-3:raise TimeoutError('Insufficient remaining real
duration')
        old_pos=self.pos.copy();old_channel=self.channel
        r=self.api.action(path,p,c)
        self.pos=np.asarray(p, dtype=float)
        self.parts['move_s']+=float(np.linalg.norm(self.pos-old_pos))/5
        if path=='/measure':
            self.channel=c;self.counts['measure']+=1
            if
r['measure_result']=='direction':self.obs[c].append((self.pos.copy(),float(r['svd_deg'])))
            self.parts['measure_s']+=5;self.parts['switch_s']+=int(c!=old_channel)
            self.measured[c].append(self.pos.copy())
        else:
            self.counts['clear']+=1;self.parts['optical_s']+=3
            if r['clear_result']=='success':self.cleared.add(c);self.parts['laser_s']+=2
            else:self.counts['failed_clear']+=1
        return r
    def clear(self,p,c):
        return self.action('/clear',p,c)['clear_result']=='success'
    def certified_clear(self,c,poly,center,radius):
        p=safe_clear_point(self.pos,center,radius)
        bound=float(np.max(np.linalg.norm(poly-p,axis=1)))
        if bound>19.990001:raise RuntimeError('Unsafe clear candidate')
        self.certificates.append({'channel':c,'vertex_bound_m':bound})
        self.diagnostics['safe_clear_points']+=1
        if not self.clear(p,c):raise RuntimeError('Certified clear failed')
    def next_station(self):
        pending=self.pending_stations
        if self.problem==4:
            inner=[i for i in pending if np.linalg.norm(self.stations[i])<=1800]
            pending=inner or pending
        return pending[open_route(self.stations[pending],self.pos)[0]]
    def process_found(self,found):
        while found:
            centers=np.array([self.region(c)[1] for c in found])
            end=self.stations[self.next_station()] if self.pending_stations else None
            c=found.pop(open_route(centers,self.pos,end=end)[0])
            self.locate(c)

```

```

def region(self, c):
    n = len(self.obs[c])
    if c not in self.cache or self.cache[c][0] != n:
        poly = feasible(self.obs[c])
        if len(poly) == 0:
            raise ValueError('Empty feasible region')
        center, radius = enclosing_circle(poly)
        self.cache[c] = (n, poly, center, radius)
    return self.cache[c][1:]

def next_measure(self, c, poly, center, radius):
    current = self.pos.copy()
    p0 = self.obs[c][-1][0]
    candidates = [current]
    for origin in (current, p0):
        d = center - origin
        length = np.linalg.norm(d)
        if length < 1e-08:
            continue
        u = d / length
        v = np.array([-u[1], u[0]])
        for fraction in (0.5, 0.8, 1.0):
            for offset in (0.0, -1.0, 1.0):
                lateral = min(100.0, max(25.0, 0.25 * length))
                candidates.append(origin + fraction * d + offset * lateral * v)
        for sign in (-1, 1):
            candidates.append(origin + 0.65 * d + sign * min(100.0, 0.35 * length) * v)
    if len(self.obs[c]) > 1:
        candidates.append((self.obs[c][0][0] + p0) / 2)
    samples = np.vstack([poly * 0.8 + center * 0.2, center[None, :]])
    if len(samples) > 9:
        samples = samples[np.linspace(0, len(samples) - 1, 9, dtype=int)]
    previous = np.array([p for p, a in self.obs[c]])
    normals = np.array([math.cos(a), math.sin(a)] for a in np.linspace(0, 2 * math.pi, 48,
endpoint=False))
    admissible = []
    for g in samples:
        mask = np.all((previous - g) @ normals.T >= -1e-07, axis=0)
        admissible.append(mask)
    best = None
    best_score = float('inf')
    seen = set()
    for q in candidates:
        key = tuple(np.round(q, 3))
        if key in seen:
            continue
        seen.add(key)
        if any((np.linalg.norm(q - p) < 0.1 for p in self.measured[c])):
            continue
        move = float(np.linalg.norm(q - current)) / 5
        radii = []
        visible = []
        for g, mask in zip(samples, admissible):
            delta = g - q
            beta = math.degrees(math.atan2(delta[1], delta[0])) % 360
            post = bearing_clip(poly, q, beta, 2 * EPS_DEG)
            if len(post):
                m = post.mean(axis=0)
                radii.append(float(np.max(np.linalg.norm(post - m, axis=1))))
            else:
                radii.append(radius)
        lower = max(1000.0, float(np.max(np.linalg.norm(previous - g, axis=1))))
        distance = float(np.linalg.norm(q - g))
        distance_weight = 1.0 if distance <= lower else max(0.0, (1500.0 - distance) /
max(1.0, 1500.0 - lower))
        if self.problem == 4 and np.any(mask):
            orientation = float(np.mean((q - g) @ normals[mask].T >= 0))
        else:
            orientation = 1.0
        visible.append(distance_weight * orientation)
    rpred = float(np.mean(radii))
    prob = float(np.mean(visible))

```

```

        remaining = float(np.mean(np.linalg.norm(samples - q, axis=1))) / 5
        score = move + 5 + int(c != self.channel) + remaining + 0.4 * rpred + 15 * (rpred > 20)
        score += (1 - prob) * (45 + move + 0.2 * radius)
        self.diagnostics['candidate_evaluations'] += 1
        if score < best_score:
            best_score = score
            best = q
    return best
def locate(self, c):
    if self.problem == 4:
        return self.locate_directional(c)
    dark = 0
    for _ in range(self.max_refine):
        poly, center, radius = self.region(c)
        if radius <= 19.99:
            self.certified_clear(c, poly, center, radius)
            return
        q = self.next_measure(c, poly, center, radius)
        if q is None:
            break
        r = self.action('/measure', q, c)
        if r['measure_result'] == 'near':
            if not self.clear(self.pos, c):
                raise RuntimeError('Near clear failed')
            return
        if r['measure_result'] == 'no_signal':
            dark += 1
            if dark >= 2:
                break
        else:
            dark = 0
    poly, center, radius = self.region(c)
    if radius <= 19.99:
        self.certified_clear(c, poly, center, radius)
        return
    self.counts['fallback'] += 1
    points = optical_cover(poly, self.obs[c][0][1])
    points = points[open_route(points, self.pos)]
    for p in points:
        if self.clear(p, c):
            return
    raise RuntimeError('Exhausted optical cover')
def locate_directional(self, c):
    if np.linalg.norm(self.pos - self.obs[c][-1][0]) > 80 and (not any((np.linalg.norm(self.pos -
    p) < 0.1 for p in self.measured[c]))):
        r = self.action('/measure', self.pos, c)
        if r['measure_result'] == 'near':
            if not self.clear(self.pos, c):
                raise RuntimeError('Near clear failed')
            return
    for _ in range(self.max_refine):
        poly, center, radius = self.region(c)
        if radius <= 19.99:
            self.certified_clear(c, poly, center, radius)
            return
        p0 = self.obs[c][-1][0]
        d = center - p0
        length = np.linalg.norm(d)
        if length < 1e-08:
            break
        v = np.array([-d[1], d[0]]) / length
        options = [p0 + 0.8 * d + sign * min(100.0, 0.35 * length) * v for sign in (-1, 1)]
        options.sort(key=lambda p: np.linalg.norm(p - self.pos))
        positive = False
        for q in options:
            r = self.action('/measure', q, c)
            if r['measure_result'] == 'near':
                if not self.clear(self.pos, c):
                    raise RuntimeError('Near clear failed')
                return
            if r['measure_result'] == 'direction':

```

```

        positive = True
        break
    if not positive:
        break
    poly, center, radius = self.region(c)
    if radius <= 19.99:
        self.certified_clear(c, poly, center, radius)
        return
    self.counts['fallback'] += 1
    points = optical_cover(poly, self.obs[c][0][1])
    points = points[open_route(points, self.pos)]
    for p in points:
        if self.clear(p, c):
            return
    raise RuntimeError('Exhausted optical cover')
def run(self):
    start = time.perf_counter()
    entered = self.api.enter()
    self.deadline = time.monotonic() + entered['remaining_real_duration_s']
    self.pending_stations = list(range(len(self.stations)))
    joint = self.problem == 3
    while self.pending_stations or self.deferred:
        self.deferred = {c: age for c, age in self.deferred.items() if c not in self.cleared}
        if len(self.cleared) == 16:
            break
        if joint and self.deferred:
            keys = list(self.deferred)
            overdue = [c for c in keys if len(self.visited) - self.deferred[c] >=
self.max_defer_stations]
            if overdue or not self.pending_stations:
                forced = overdue or keys
                c = min(forced, key=lambda c: np.linalg.norm(self.region(c)[1] - self.pos))
                self.diagnostics['forced_targets'] += 1
                self.locate(c)
                self.deferred.pop(c, None)
                continue
            nodes = [('target', c) for c in keys] + [('station', j) for j in
self.pending_stations]
            points = np.array([self.region(c)[1] for c in keys] + [self.stations[j] for j in
self.pending_stations])
            kind, which = nodes[open_route(points, self.pos)[0]]
            if kind == 'target':
                self.locate(which)
                self.deferred.pop(which, None)
                continue
            i = which
        elif self.pending_stations:
            i = self.next_station()
        else:
            break
        self.pending_stations.remove(i)
        p = self.stations[i]
        found = []
        channels = [c for c in range(1, 21) if c not in self.cleared]
        if self.channel in channels:
            channels.remove(self.channel)
            channels.insert(0, self.channel)
        for c in channels:
            r = self.action('/measure', p, c)
            if r['measure_result'] == 'near':
                if not self.clear(p, c):
                    raise RuntimeError('Near clear failed')
            elif r['measure_result'] == 'direction':
                found.append(c)
            if c in self.deferred:
                self.diagnostics['opportunistic_measures'] += 1
        self.visited.append(i)
        if joint:
            for c in found:
                if c not in self.deferred:
                    self.deferred[c] = len(self.visited)

```

```

        self.diagnostics['deferred_targets'] += 1
    else:
        self.process_found(found)
        if len(self.cleared) == 16:
            break
        if len(self.cleared) < 16 and (self.pending_stations or any((c not in self.cleared for c in
self.deferred))):
            raise RuntimeError('Cannot finish with unresolved coverage or targets')
        exited = self.api.exit()
        if exited.get('accepted') is not True or exited.get('exit_reason') != 'user_exit':
            raise RuntimeError('Normal exit not acknowledged')
        total = self.api.virtual_time
        return dict(problem=self.problem, cleared=len(self.cleared), virtual_time_s=total,
mean_time_s=total / len(self.cleared) if self.cleared else None,
program_time_s=time.perf_counter() - start, complete=True, stations_visited=len(self.visited),
**self.counts, **self.parts, **self.diagnostics)

```

backend.py

```

"""Official HTTP adapter and independently generated local scenarios.
The controller has no access to scenario truth. Offline outcomes are not official.
"""
import hashlib
import http.client
import json
import math
import time
import uuid
import urllib.request
import urllib.error
from pathlib import Path
import numpy as np
class HTTPBackend:
    def __init__(self, robot_id, url, log_path):
        self.robot_id=robot_id
        self.url=url.rstrip('/')
        self.virtual_time=0.
        self.log=Path(log_path).open('a', encoding='utf-8')
        self.deadline=time.monotonic()+60
        self.session=uuid.uuid4().hex[:12]
        self.seq=0
        self.opener=urllib.request.build_opener(urllib.request.ProxyHandler({}))
    def post(self, path, payload):
        self.seq+=1
        body={'arena_id':'default', 'robot_id':self.robot_id,
            'request_id':f'{self.session}-{self.seq}', **payload}
        data=json.dumps(body, ensure_ascii=False, allow_nan=False, separators=(',', ':')).encode('utf-8')
        # Retries preserve byte-identical body, request ID and endpoint.
        for attempt in range(6):
            if time.monotonic()>=self.deadline:
                raise TimeoutError('HTTP deadline reached')
            request=urllib.request.Request(self.url+path, data=data,
                headers={'Content-Type':'application/json; charset=utf-8'}, method='POST')
            try:
                with self.opener.open(request, timeout=min(5, max(.1, self.deadline-time.monotonic())))
as response:
                    result=json.load(response)
self.log.write(json.dumps({'path':path, 'request':body, 'response':result}, ensure_ascii=False)+'\n'
)
                self.log.flush()
                if result.get('accepted') is not True:
                    raise RuntimeError('Request rejected; check robot_id and simulator state')
                self.virtual_time=float(result['virtual_time_s'])
                return result
        except urllib.error.HTTPError as e:
            # Error replies are not silently turned into new actions.
            self.log.write(json.dumps({'path':path, 'request':body, 'http_error':e.code})+'\n')
            self.log.flush()
            raise

```

```

        except (urllib.error.URLError,TimeoutError,ConnectionError,
                http.client.IncompleteRead,json.JSONDecodeError) as e:
            self.log.write(json.dumps({'path':path,'request':body,'transport_error':str(e),'attempt':attempt})+'\n')
            self.log.flush()
            if attempt==5:
                raise
            time.sleep(min(.25*2**attempt,2.))
    def enter(self):
        r=self.post('/enter',{})
        self.deadline=time.monotonic()+r['remaining_real_duration_s']
        return r
    def action(self,path,p,c):
        return self.post(path,{'position':{'x':float(p[0]),'y':float(p[1]),'channel':int(c)})
    def exit(self):
        return self.post('/exit',{})
class LocalBackend:
    def __init__(self,seed,problem,error_mode='fixed',radius=None,n=None,all_directional=False):
        rng=np.random.default_rng(seed)
        self.seed=seed
        self.error_mode=error_mode
        count=int(rng.integers(10,17)) if n is None else n
        channels=rng.choice(np.arange(1,21),count,replace=False)
        self.sources={}
        for c in channels:
            r=1800*math.sqrt(float(rng.random())); a=float(rng.uniform(0,2*math.pi))
            self.sources[int(c)]={'p':np.array([r*math.cos(a),r*math.sin(a)]),
                                'radius':float(rng.uniform(1000,1500)) if radius is None else radius,
                                'direction':float(rng.uniform(0,2*math.pi)) if problem==4 and (all_directional or
rng.random()<.5) else None,
                                'alive':True}
            self.pos=np.zeros(2); self.channel=1; self.virtual_time=0.; self.trace=[]
            self.parts={'move_s':0.,'switch_s':0.,'measure_s':0.,'optical_s':0.,'laser_s':0.}
    def enter(self):
        return {'accepted':True,'remaining_real_duration_s':1200,'virtual_time_s':0.}
    def error(self,p,c):
        if self.error_mode=='plus':
            return 1.
        if self.error_mode=='minus':
            return -1.
        # Deterministic by position/channel: no fresh noise on repeated readings.
        text=f'{self.seed}:{c}:{p[0]:.8f}:{p[1]:.8f}'.encode()
        h=int.from_bytes(hashlib.sha256(text).digest()[:8],'big')
        return 2*h/(2**64-1)-1
    def action(self,path,p,c):
        p=np.asarray(p,dtype=float)
        move=float(np.linalg.norm(p-self.pos))/5
        self.parts['move_s']+=move
        self.virtual_time+=move
        self.pos=p.copy()
        s=self.sources.get(c)
        distance=float(np.linalg.norm(p-s['p'])) if s else float('inf')
        result={'accepted':True}
        if path=='/measure':
            switch=int(self.channel!=c)
            self.parts['switch_s']+=switch; self.parts['measure_s']+=5
            self.virtual_time+=5+switch
            self.channel=c
            visible=bool(s and s['alive'] and distance<=s['radius'])
            if visible and s['direction'] is not None:
                v=np.array([math.cos(s['direction']),math.sin(s['direction'])])
                visible=float((p-s['p'])@v)>=-1e-9
            if not visible:
                result['measure_result']='no_signal'
            elif distance<=5:
                result['measure_result']='near'
            else:
                result['measure_result']='direction'
                delta=s['p']-p
                angle=(math.degrees(math.atan2(delta[1],delta[0]))+self.error(p,c))%360
                result['svd_deg']=round(angle,2)%360

```

```

else:
    self.parts['optical_s']+=3; self.virtual_time+=3
    success=bool(s and s['alive'] and distance<=20)
    result['clear_result']='success' if success else 'no_target_in_range'
    if success:
        s['alive']=False
        self.parts['laser_s']+=2; self.virtual_time+=2
    result['virtual_time_s']=self.virtual_time
    self.trace.append({'path':path, 'p':p.tolist(), 'channel':int(c), 'result':result.copy()})
    if self.virtual_time>=360000:
        raise TimeoutError('Virtual time limit')
    return result
def exit(self):
    return {'accepted':True, 'virtual_time_s':self.virtual_time, 'exit_reason':'user_exit'}

```

audit.py

```

"""Protocol-only port with independent action/time accounting.
Controllers receive this surface, which exposes no source coordinates or counts.
"""
import math
import time
class AuditPort:
    def __init__(self, backend):
        self.__backend=backend
        self.virtual_time=0.
        self.position=(0.,0.)
        self.channel=1
        self.entered=False
        self.exited=False
        self.success_channels=set()
        self.parts={'move_s':0., 'switch_s':0., 'measure_s':0., 'optical_s':0., 'laser_s':0.}
        self.max_step_error_s=0.
        self.actions=0
        self.deadline=None
    def enter(self):
        if self.entered:raise RuntimeError('Duplicate enter')
        r=self.__backend.enter()
        if r.get('accepted') is not True:raise RuntimeError('Enter rejected')
        self.entered=True
        self.virtual_time=float(r['virtual_time_s'])
        self.deadline=time.monotonic()+float(r['remaining_real_duration_s'])
        return r
    def action(self, path, p, c):
        if not self.entered or self.exited:raise RuntimeError('Action outside session')
        if time.monotonic()>=self.deadline:raise TimeoutError('Real deadline reached')
        if path not in ('/measure', '/clear'):raise ValueError('Unknown action')
        if not isinstance(c, int) or not 1<=c<=20:raise ValueError('Invalid channel')
        p=tuple(float(x) for x in p)
        if len(p)!=2 or not all(math.isfinite(x) and abs(x)<=2000000 for x in p):
            raise ValueError('Invalid coordinates')
        r=self.__backend.action(path, p, c)
        if r.get('accepted') is not True:raise RuntimeError('Action rejected')
        delta=dict.fromkeys(self.parts, 0.)
        delta['move_s']=math.dist(p, self.position)/5
        if path=='/measure':
            kind=r.get('measure_result')
            if kind not in ('direction', 'near', 'no_signal'):raise RuntimeError('Invalid measure result')
            if kind=='direction' and not (math.isfinite(r['svd_deg']) and 0<=r['svd_deg']<360):
                raise RuntimeError('Invalid bearing')
            delta['measure_s']=5
            delta['switch_s']=int(self.channel!=c)
            self.channel=c
        else:
            kind=r.get('clear_result')
            if kind not in ('success', 'no_target_in_range'):raise RuntimeError('Invalid clear result')
            delta['optical_s']=3

```

```

        if kind=='success':
            if c in self.success_channels:raise RuntimeError('Duplicate source clearance')
            self.success_channels.add(c);delta['laser_s']=2
            error=abs(float(r['virtual_time_s'])-(self.virtual_time+sum(delta.values()))))
            self.max_step_error_s=max(self.max_step_error_s,error)
            # Protocol uses rounded cumulative microseconds.
            if error>1e-4:raise RuntimeError('Simulator timing differs from attachment rules')
            for k,v in delta.items():self.parts[k]+=v
            self.position=p;self.virtual_time=float(r['virtual_time_s']);self.actions+=1
            return r
def exit(self):
    if not self.entered or self.exited:raise RuntimeError('Invalid exit')
    r=self.__backend.exit()
    if r.get('accepted') is not True or r.get('exit_reason')!='user_exit':
        raise RuntimeError('Normal exit not acknowledged')
    if abs(float(r['virtual_time_s'])-self.virtual_time)>1e-6:
        raise RuntimeError('Exit changed virtual time')
    self.exited=True
    return r

```

run. py

```

"""One entry point for the maintained controller and local experiments."""
import argparse, csv, json, time
from pathlib import Path
from backend import HTTPBackend, LocalBackend
from solver import Solver
from audit import AuditPort
ROOT=Path(__file__).resolve().parent
def main():
    ap=argparse.ArgumentParser(description='Radio-source search and clearance')
    ap.add_argument('--mode', choices=['offline', 'official'], default='offline')
    ap.add_argument('--problem', type=int, choices=[3, 4], default=3)
    ap.add_argument('--robot-id'); ap.add_argument('--url', default='http://127.0.0.1:2026')
    ap.add_argument('--runs', type=int, default=1); ap.add_argument('--seed', type=int, default=20260910)
    ap.add_argument('--spacing', type=float, default=950.); ap.add_argument('--
    refine', type=int, default=8)
    ap.add_argument('--error', choices=['fixed', 'plus', 'minus'], default='fixed')
    ap.add_argument('--radius', type=float); ap.add_argument('--all-directional', action='store_true')
    ap.add_argument('--output', default=None)
    a=ap.parse_args()
    if a.mode=='official' and not a.robot_id: ap.error('--robot-id is required')
    if a.runs<1: ap.error('--runs must be positive')
    if a.mode=='official' and (a.runs!=1 or a.spacing!=950 or a.refine!=8):
        ap.error('Official runs use the maintained default policy; tuning flags are offline only')
    out=Path(a.output) if a.output else ROOT/('practice_logs' if a.mode=='official' else 'results')
    out.mkdir(parents=True, exist_ok=True)
    stamp=time.strftime('%Y%m%d_%H%M%S')+ '_' +str(time.time_ns()%1000000000)
    rows=[]
    for i in range(1 if a.mode=='official' else a.runs):
        stem=out/f'client_q{a.problem}_{stamp}_{i+1}'
        backend=(HTTPBackend(a.robot_id, a.url, stem.with_suffix('.jsonl')) if a.mode=='official'
                else
                LocalBackend(a.seed+i, a.problem, a.error, a.radius, all_directional=a.all_directional))
        port=AuditPort(backend)
        try:
            r=Solver(port, a.problem, a.spacing, a.refine).run()
            r.update(normal_exit=port.exited, max_step_time_error_s=port.max_step_error_s, **port.parts)
            r['provenance']='Official HTTP interface; case code and module come from simulator UI'
            if a.mode=='offline':
                r.update(seed=a.seed+i, total=len(backend.sources), ratio=r['cleared']/len(backend.sources),
                        provenance='LOCAL_SYNTHETIC', spacing=a.spacing, error=a.error,
                        directional=sum(s['direction'] is not None for s in
                        backend.sources.values()))
            assert r['cleared']==len(backend.sources)
            if i==0:
                truth={k:{v.tolist() if hasattr(v, 'tolist') else v} for k, v in s.items()} for
                c, s in backend.sources.items()}
                (out/f'local_q{a.problem}_example.json').write_text(json.dumps({'sources': truth, '

```

```

        trace':backend.trace},indent=2),encoding='utf-8')
        else:stem.with_suffix('.json').write_text(json.dumps(r,indent=2),encoding='utf-8')
        rows.append(r)
        if a.mode=='official' or a.runs==1:print(json.dumps(r,indent=2),flush=True)
        else:print(f'LOCAL Q{a.problem} seed={a.seed+i} cleared={r["cleared"]}/{r["total"]}')
mean={r["mean_time_s"]:.3f}',flush=True)
    except Exception as exc:
        failed={'complete':False,'normal_exit':port.exited,'problem':a.problem,
                'last_acked_virtual_time_s':port.virtual_time,
                'error':type(exc).__name__+' '+str(exc)}
        stem.with_suffix('.error.json').write_text(json.dumps(failed,indent=2),encoding='utf-8')
        raise
    finally:
        if hasattr(backend,'log'):backend.log.close()
if a.mode=='offline':
    with (out/f'local_q{a.problem}.csv').open('w',newline='',encoding='utf-8-sig') as f:
        w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)
if __name__=='__main__':main()

```

question2. py

```

"""Q2: safe candidate lens and discrete minimax linearized precision score.
Normalized first observation S=(0,0), measured bearing=0 degrees.
Transform candidates by rotation/translation for any actual observation.
"""
import csv
import json
import math
from pathlib import Path
import numpy as np
from geometry import unit,EPS_DEG,region_from_bearings,enclosing_circle
def main():
    out=Path('results');out.mkdir(exist_ok=True)
    centers=np.array([[0.,0.],1000*unit(-EPS_DEG),1000*unit(EPS_DEG)])
    sources=np.array([r*unit(a) for r in np.linspace(5,1500,151)
                      for a in np.linspace(-EPS_DEG,EPS_DEG,9)])
    r1=np.linalg.norm(sources,axis=1)
    rows=[]
    for a in range(25,1001,25):
        for b in range(25,1001,25):
            q=np.array([a,b])
            if np.max(np.linalg.norm(centers-q,axis=1))>1000-1e-6:
                continue
            d=sources-q
            r2=np.linalg.norm(d,axis=1)
            sin=np.abs(sources[:,0]*d[:,1]-sources[:,1]*d[:,0])/(r1*r2)
            # sigma=1 degree/sqrt(3) is a comparative assumption, not a fact.
            score=float(np.sqrt(r1*r1+r2*r2)/np.maximum(sin,1e-12))
            rows.append(dict(a_m=a,b_m=b,score_per_radian=score))
    rows.sort(key=lambda x:x['score_per_radian'])
    with (out/'question2_candidates.csv').open('w',newline='',encoding='utf-8-sig') as f:
        w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)
    q1=region_from_bearings([(0,0),45],([1000,0],135))
    c,r=enclosing_circle(q1['vertices'])
    summary={'q2_discrete_best':rows[0],'q2_candidates_upper_half':len(rows),
            'q2_uniform_noise_assumption_rms_m':rows[0]['score_per_radian']*math.radians(1)/math.sqrt(3),
            'q1_example':{'diameter':q1['diameter'],'circle_center':c.tolist(),'circle_radius':r,'vertices':q1['vertices'].tolist()}}
    (out/'geometry_summary.json').write_text(json.dumps(summary,indent=2),encoding='utf-8')
    print(json.dumps(summary,indent=2))
if __name__=='__main__':
    main()

```

experiments. py

```

"""Reproducible stress and sensitivity experiments; never official results."""
import csv
import json

```

```

from pathlib import Path
import numpy as np
from backend import LocalBackend
from solver import Solver
from audit import AuditPort
def run_batch(out,name,problem,seeds,**kwargs):
    rows=[]
    for seed in seeds:
        local={k:kwargs[k] for k in ['error_mode','radius','all_directional'] if k in kwargs}
        b=LocalBackend(seed,problem,**local)
        # Additional deterministic boundary/tangent scenarios.
        if name.startswith('boundary'):
            for i,s in enumerate(b.sources.values()):
                angle=2*np.pi*i/len(b.sources)
                s['p']=1800*np.array([np.cos(angle),np.sin(angle)])
                s['radius']=1000.
                s['direction']=angle if seed%2 else angle+np.pi/2
            solve={k:kwargs[k] for k in ['spacing','max_refine'] if k in kwargs}
            port=AuditPort(b)
            r=Solver(port,problem,**solve).run()
            r.update(seed=seed,total=len(b.sources),ratio=r['cleared']/len(b.sources),
                    experiment=name,provenance='LOCAL_SYNTHETIC',normal_exit=port.exited,**b.parts)
            assert r['ratio']==1,r
            rows.append(r)
    with (out/(name+'.csv')).open('w',newline='',encoding='utf-8-sig') as f:
        w=csv.DictWriter(f,fieldnames=list(rows[0]));w.writeheader();w.writerows(rows)
    result=dict(experiment=name,runs=len(rows),cleared=sum(r['cleared'] for r in rows),
               total=sum(r['total'] for r in rows),mean_time_s=float(np.mean([r['mean_time_s'] for r in
               rows])),
               weighted_mean_s=sum(r['virtual_time_s'] for r in rows)/sum(r['cleared'] for r in rows),
               max_mean_s=max(r['mean_time_s'] for r in rows),fallback=sum(r['fallback'] for r in rows))
    print(json.dumps(result),flush=True)
    return result
def main():
    out=Path('results/stress');out.mkdir(parents=True,exist_ok=True)
    summary=[]
    for p in (3,4):
        for error in ('plus','minus'):
            summary.append(run_batch(out,f'q{p}_{error}_r1000',p,range(100,120),
                                   error_mode=error,radius=1000.,all_directional=True))
    summary.append(run_batch(out,'boundary_outward_tangent',4,range(20,40),all_directional=True))
    for h in (800,900,950,999):
        summary.append(run_batch(out,f'q4_spacing_{h}',4,range(200,220),spacing=h))
    for p in (3,4):
        summary.append(run_batch(out,f'q{p}_optical_only',p,range(20260910,20260920),max_refine=0))
    (out/'summary.json').write_text(json.dumps(summary,indent=2),encoding='utf-8')
if __name__=='__main__':
    main()

```

test_model.py

```

import math
import io
import json
import tempfile
import unittest
import urllib.error
from pathlib import Path
from unittest.mock import patch
import numpy as np
from geometry import (unit,feasible,enclosing_circle,diameter,region_from_bearings,
                     search_stations,optical_cover)
from backend import LocalBackend,HTTPBackend
from solver import Solver
class ModelTests(unittest.TestCase):
    def test_attachment_timing(self):
        b=LocalBackend(1,3)
        b.sources={}
        b.enter()

```

```

        b.action('/measure', [300, 400], 1)
        b.action('/measure', [300, 400], 2)
        b.action('/clear', [300, 0], 3)
        b.action('/measure', [300, 0], 2)
        self.assertEqual(b.virtual_time, 199)
        self.assertEqual(b.channel, 2)
    def test_equilateral(self):
        p=np.array([[0., 0.], [40., 0.], [20., 20*math.sqrt(3)]]))
        self.assertAlmostEqual(diameter(p)[0], 40)
        self.assertAlmostEqual(enclosing_circle(p)[1], 40/math.sqrt(3))
        self.assertGreater(enclosing_circle(p)[1], 20)
    def test_bounded_empty_unbounded(self):
        self.assertEqual(region_from_bearings([(0, 0), 0])[0]['status'], 'unbounded')
        self.assertEqual(region_from_bearings([(0, 0), 180], ([1, 0], 0))[0]['status'], 'empty')
        r=region_from_bearings([(0, 0), 45], ([1000, 0], 135))
        self.assertEqual(r['status'], 'bounded')
        self.assertTrue(0<r['diameter']<100)
    def test_rounding_wrap_and_region_contains_truth(self):
        g=np.array([1200., -.01])
        observations=[]
        for p,e in [(np.array([0., 0.]), 1.005), (np.array([1000., 300.]), -1.005)]:
            d=g-p
            theta=(math.degrees(math.atan2(d[1], d[0]))+e)%360
            observations.append((p, theta))
        poly=feasible(observations)
        for a,b in zip(poly, np.roll(poly, -1, axis=0)):
            u=b-a; v=g-a
            self.assertGreaterEqual(float(u[0]*v[1]-u[1]*v[0]), -1e-5)
    def test_repeated_position_error_is_fixed(self):
        b=LocalBackend(3, 3)
        self.assertEqual(b.error([10., 20.], 1), b.error([10., 20.], 1))
    def test_q3_geometric_coverage(self):
        s=search_stations(3)
        for r in np.linspace(0, 1800, 61):
            for angle in np.linspace(0, 360, 361):
                self.assertLessEqual(np.min(np.linalg.norm(s-r*unit(angle), axis=1)), 1000+1e-6)
    def test_q4_halfplane_coverage_boundary_and_interior(self):
        s=search_stations(4)
        rng=np.random.default_rng(10)
        points=[1800*unit(a) for a in np.arange(0, 360, 3.)]
        points.extend(1800*math.sqrt(rng.random())*unit(rng.uniform(0, 360)) for _ in range(400))
        for p in points:
            delta=s-p
            nearby=delta[np.linalg.norm(delta, axis=1)<=1000+1e-6]
            for angle in range(0, 360, 5):
                self.assertTrue(np.any(nearby@unit(angle)>=-1e-6))
    def test_optical_cover(self):
        poly=feasible([(np.array([0., 0.]), 359.99)])
        points=optical_cover(poly, 359.99)
        rng=np.random.default_rng(1)
        for _ in range(1000):
            weights=rng.random(len(poly)); weights/=weights.sum()
            p=weights@poly
            self.assertLessEqual(np.min(np.linalg.norm(points-p, axis=1)), 20.)
    def test_clear_independent_of_beam(self):
        b=LocalBackend(3, 4)
        b.sources={1:dict(p=np.array([0., 0.]), radius=1000., direction=0., alive=True)}
        self.assertEqual(b.action('/measure', [-10, 0], 1)['measure_result'], 'no_signal')
        self.assertEqual(b.action('/clear', [-10, 0], 1)['clear_result'], 'success')
        self.assertEqual(b.action('/clear', [-10, 0], 1)['clear_result'], 'no_target_in_range')
    def test_optical_fallback_forced(self):
        b=LocalBackend(11, 4, all_directional=True, n=10, radius=1000.)
        result=Solver(b, 4, max_refine=0).run()
        self.assertEqual(result['cleared'], 10)
        self.assertGreater(result['fallback'], 0)
    def test_http_retry_idempotency_and_rejection(self):
        class FakeOpener:
            def __init__(self):
                self.requests=[]
            def open(self, request, timeout):
                self.requests.append(request)

```

```

        if len(self.requests)==1:
            raise urllib.error.URLError('response lost after execution')
        return io.BytesIO(json.dumps({'accepted':True, 'virtual_time_s':5}).encode())
    with tempfile.TemporaryDirectory() as tmp:
        api=HTTPBackend('TEST', 'http://127.0.0.1:2026', Path(tmp)/'log.jsonl')
        fake=FakeOpener();api.opener=fake
        with patch('backend.time.sleep'):
            api.action('/measure', [0,0],1)
            self.assertEqual(fake.requests[0].data, fake.requests[1].data)
            self.assertEqual(fake.requests[0].full_url, fake.requests[1].full_url)
            self.assertEqual(api.virtual_time,5)
            api.log.close()
if __name__=='__main__':
    unittest.main(verbosity=2)

```

test_controller.py

```

import unittest
import numpy as np
from audit import AuditPort
from backend import LocalBackend
from solver import Solver, safe_clear_point, open_route
class ControllerTests(unittest.TestCase):
    def test_safe_points_cover_entire_region(self):
        for r in (0,5,19,19.99):
            poly=np.array([[r,0],[0,r],[-r,0],[0,-r]])
            for position in ([100,20],[0,0],[-30,10]):
                q=safe_clear_point(position,np.zeros(2),r)
                self.assertLessEqual(np.max(np.linalg.norm(poly-q, axis=1)),19.990001)
    def test_open_route_preserves_all_tasks(self):
        points=np.random.default_rng(75).normal(size=(24,2))*1000
        order=open_route(points,np.array([0,0]))
        self.assertEqual(sorted(order), list(range(24)))
    def test_ten_sources_needs_complete_coverage(self):
        for problem in (3,4):
            b=LocalBackend(800001, problem, n=10, all_directional=True, radius=1000.)
            port=AuditPort(b);s=Solver(port, problem);r=s.run()
            self.assertEqual(r['cleared'],10)
            self.assertEqual(r['stations_visited'], len(s.stations))
            self.assertTrue(port.exited)
            self.assertFalse(any(c not in s.cleared for c in s.deferred))
    def test_no_new_action_after_unknown_failure(self):
        class Broken:
            virtual_time=0.;exited=False
            def enter(self):return
        {'accepted':True, 'virtual_time_s':0., 'remaining_real_duration_s':1200}
        def action(self, *args):raise ConnectionError('Response unknown')
        def exit(self):self.exited=True
        b=Broken()
        with self.assertRaises(ConnectionError):Solver(b,3).run()
        self.assertFalse(b.exited)
    def test_audit_requires_valid_exit_and_has_no_truth_interface(self):
        b=LocalBackend(2,3);b.sources={};a=AuditPort(b);a.enter()
        a.action('/measure', [300,400],1);a.action('/measure', [300,400],2)
        a.action('/clear', [300,0],3);a.action('/measure', [300,0],2);a.exit()
        self.assertEqual(a.virtual_time,199)
        with self.assertRaises(AttributeError):getattr(a, 'sources')
        with self.assertRaises(RuntimeError):a.action('/measure', [0,0],1)
if __name__=='__main__':unittest.main()

```

verify_experiments.py

```

"""Reproduce the comparison in a temporary directory, not the live controller."""
import argparse, base64, json, subprocess, sys, tempfile, shutil
from pathlib import Path
def main():
    ap=argparse.ArgumentParser(description=__doc__)
    ap.add_argument('--problem', type=int, choices=[3,4], required=True)

```

```

ap.add_argument('--count', type=int, default=200)
ap.add_argument('--output', type=Path, default=Path('results/reproduction'))
a=ap.parse_args()
if not 1<=a.count<=200:ap.error('--count must be between 1 and 200')
root=Path(__file__).resolve().parent/'results/optimization'
fixture=json.loads((root/'experiment_sources.json').read_text(encoding='utf-8'))
with tempfile.TemporaryDirectory(prefix='source-comparison-') as tmp:
    path=Path(tmp)
    for name,data in fixture['files'].items():
        if Path(name).name!=name or not name.endswith('.py'):raise ValueError('Invalid fixture
path')
        (path/name).write_bytes(base64.b64decode(data['bytes_base64']))
        shutil.copy2(root/'protocol.json', path/'protocol.json')
        mode='combined' if a.problem==3 else 'route'
        subprocess.run([sys.executable, 'evaluate.py', '--problem', str(a.problem), '--start', '820000',
                        '--count', str(a.count), '--modes', 'original', 'reference', mode, '--
tag', 'reproduction'], cwd=path, check=True)
        a.output.mkdir(parents=True, exist_ok=True)
        shutil.copy2(path/f'results/reproduction/q{a.problem}.csv', a.output/f'q{a.problem}.csv')
if __name__=='__main__':main()

```